



## Mission - Développez un assistant pour la recommandation d'événements culturels



### Comment allez-vous procéder ?



Cette mission suit un scénario de projet professionnel.

Vous pouvez suivre les étapes pour vous aider à réaliser vos livrables.

Avant de démarrer, nous vous conseillons de :

- lire toute la mission et ses documents liés ;
- prendre des notes sur ce que vous avez compris ;
- consulter les étapes pour vous guider ;
- préparer une liste de questions pour votre première session de mentorat.

### Prêt à mener la mission ?



Vous êtes ingénieur data freelance spécialisé en intégration de modèles NLP et gestion de bases de données vectorielles. Vous travaillez pour l'entreprise Puls-Events, une société innovante dans le domaine de la gestion d'événements culturels.

L'entreprise Puls-Events propose une plateforme web permettant aux utilisateurs de découvrir et de suivre des événements culturels en temps réel. Elle collecte des informations à partir de diverses sources, comme Open Agenda, pour proposer des événements adaptés aux préférences de ses utilisateurs, filtrables par lieu et par période.

Elle souhaite tester l'intégration d'un chatbot intelligent, capable de fournir des recommandations personnalisées et augmentées par des réponses précises issues de données d'événements.

Ce chatbot, soutenu par un système de génération de réponses augmentées (RAG), doit être capable **d'extraire** et **d'analyser** des informations à partir d'une base de données vectorielle.

Vous êtes chargé de réaliser un premier Proof of Concept (POC) pour démontrer la faisabilité de ce concept.

Vous recevez un message de votre responsable technique :



**De** : Jérémie

**A** : moi

**Objet** : version fonctionnelle du POC RAG

Salut,

Nous avons besoin que tu termines une version fonctionnelle du POC RAG. Il sera présenté en réunion avec les équipes produit et marketing pour vérifier l'intérêt du projet et envisager son déploiement à plus grande échelle.

Comme discuté ensemble, nous allons nous appuyer sur la source de données des [événements publics Open Agenda](#). Tu pourras te limiter au **périmètre géographique de ton choix**. Veille cependant à ce qu'il y ait uniquement des **événements récents de moins d'un an**.

**Ce qui est attendu :**

- **Environnement virtuel** : Pour que le projet soit facilement reproductible, dépendances et documentation sous forme d'un readme.
- **Système RAG fonctionnel** : Code complet du système RAG versionné dans un repo git, avec intégration de LangChain, Mistral et Faiss. Le système doit être en mesure de reconstruire la base vectorielle sur demande (scripts de pré-processing et de vectorisation des données).
- **Rapport technique (PDF ou Word)** : Document entre 5 et 10 pages, expliquant l'architecture du système, tes choix techniques, les modèles sélectionnés, les résultats obtenus pour le POC et les recommandations pour la version finale.
- **Tests unitaires (Python)** : Prévois un fichier .py pour tester que les données intégrées dans la base vectorielle correspondent bien à des événements de moins d'un an dans la région géographique que tu auras sélectionnée.
- **Présentation (PowerPoint)** : Entre 10 et 15 slides pour exposer le concept du projet, les résultats du POC.
- **Démo live du système** : Prépare une démo live pour illustrer les capacités du POC.

N'hésite pas si tu as besoin de précisions sur le périmètre ou les attentes. On compte sur toi pour que tout soit prêt dans les délais !

Ah oui, j'oubliais..Pour évaluer le système, nous aurons besoin de créer un jeu de données test annoté de questions / réponses. Ce jeu test sera utilisé pour mesurer la qualité des réponses par rapport aux réponses annotées.

Merci et bon courage !

Jérémie

Cette mission est entièrement guidée. Vous pouvez suivre les étapes ci-dessous.

## Étapes

### Étape 1 - Préparez l'environnement de travail

Ce projet situé vers la fin du parcours ne contient pas de fiche d'autoévaluation

#### Description

- Configurer l'environnement de développement nécessaire pour implémenter le système RAG.
- Installer les bibliothèques requises,
- et s'assurer que tout fonctionne correctement.



• Avoir installé les bibliothèques LangChain et Faiss.

### Résultat attendu

Un environnement de développement fonctionnel avec tous les outils installés et prêts à être utilisés.

### Recommandations

- Commencez par créer un environnement virtuel.
- Installez les packages requis : `pip install langchain faiss-cpu mistral`.
- Testez que toutes les installations sont correctes en exécutant un simple script Python pour importer les bibliothèques requises.

### Points de vigilance

- Vérifier que toutes les versions des packages sont compatibles.
- S'assurer que Faiss est bien installé avec le backend CPU pour éviter des erreurs.

### Outils

- Python 3
- LangChain, Faiss, Mistral API
- Environnement virtuel (e.g., virtualenv ou conda)

### Ressources

- Documentation Faiss
- LangChain API Reference

## Étape 2 - Effectuez le pré-processing des données Open Agenda



### Description

Récupérer les données d'événements à partir de la plateforme Open Agenda, les filtrer par localisation et période, et structurer ces données pour une utilisation future dans la base de données vectorielle.

### Prérequis

- Avoir accès à l'API Open Agenda.
- Avoir compris les paramètres de filtrage souhaités (ville, période, type d'événement).

### Résultat attendu

Un jeu de données d'événements propre et structuré, prêt à être indexé. Vous pourrez effectuer des tests unitaires pour vous assurer d'avoir récupéré les données attendues.

### Recommandations

- Utiliser l'API Open Agenda pour récupérer les événements.
- Filtrer les événements par localisation et période (1 an d'historique et événements à venir).
- Convertir les descriptions des événements en format vectoriel avec un modèle de NLP comme Mistral.

### Points de vigilance



### Outils

- OpenAgenda API
- Pandas pour la manipulation des données
- Mistral pour la génération de vecteurs

### Ressources

- [OpenAgenda API Documentation](#)
- Mistral Model Documentation

## Étape 3 - Implémentez la base de données vectorielle avec Faiss



### Description

Indexer les descriptions des événements sous forme de vecteurs dans une base de données vectorielle avec Faiss, permettant une recherche rapide basée sur la similarité sémantique.

### Prérequis

- Avoir nettoyé les données d'Open Agenda.
- Avoir découpé les textes en chunks avant la vectorisation des textes.
- Vectorisation de chaque chunk et indexation dans la base de données FAISS

### Résultat attendu

Une base de données Faiss contenant les événements indexés en fonction de leurs vecteurs sémantiques.

### Recommandations

- Utilisez Faiss pour construire un index de recherche sémantique.
- Enregistrez les informations non seulement sur les vecteurs, mais aussi sur les métadonnées des événements (dates, lieux, descriptions).
- Faites des tests de recherche pour vérifier l'efficacité.

### Points de vigilance

- S'assurer que l'index est optimisé pour des recherches rapides (utiliser les bons algorithmes dans Faiss).
- Vérifier que tous les événements ont bien été indexés.

### Outils

- Faiss pour l'indexation vectorielle
- LangChain pour interagir avec Faiss

### Ressource

- [Faiss Index Guide](#)

## Étape 4 - Intégrez LangChain pour le système RAG



### Description



Développez un chatbot intelligent capable de fournir des recommandations personnalisées basées sur les événements indexés, et générer des réponses augmentées à partir des données présentes dans la base de données vectorielle.

### Prérequis

- Les événements sont indexés dans Faiss.
- API Mistral pour générer des réponses naturelles à partir d'un LLM sélectionné sur la [plateforme Mistral](#).

### Résultat attendu

Un chatbot capable de fournir des recommandations d'événements et d'interagir avec l'utilisateur.

Une réponse IA est correcte si elle a le même sens et contient les mêmes informations que la réponse annotée par l'humain.

### Recommandations

- Utilisez LangChain pour orchestrer les appels entre la base Faiss et le modèle Mistral.
- Configurez des chaînes de traitement pour que le chatbot puisse générer des réponses basées à la fois sur la recherche dans Faiss et sur le modèle de langage.

### Points de vigilance

- S'assurer que les réponses générées sont à la fois pertinentes et bien formulées.
- Tester plusieurs scénarios d'interaction pour vérifier la robustesse du chatbot.
- L'historique de conversation entre le bot et l'humain n'est pas nécessaire dans le POC.

### Outils

- LangChain pour la gestion des interactions
- Mistral pour la génération de réponses

### Ressources

- LangChain Documentation
- Mistral Model

! [Avez-vous une suggestion pour nous ?](#) ↗

[Précédent](#)[Suivant](#)

